The Developer, the RFC, and the Middlebox: An HTTP/2 Compliance Story

Mahmoud Attia
mahmoud.attia@kaust.edu.sa
KAUST
Thuwal, Saudi Arabia

Ilies Benhabbour
ilies.benhabbour@kaust.edu.sa
KAUST
Thuwal, Saudi Arabia

Marc Dacier
marc.dacier@kaust.edu.sa
KAUST
Thuwal, Saudi Arabia

Abstract

The current Internet landscape is shaped by intermediate elements, known as middleboxes, which disrupt the end-to-end principle the Internet was originally designed around. In [5], the authors examine the HTTP/1.1 protocol and demonstrate that assessing the compliance of a remote implementation with respect to its RFC is challenging, as middleboxes on the network path can alter application-layer content. This paper builds on their work by focusing on HTTP/2, a more efficient yet more complex protocol that is known to be vulnerable to denial-of-service attacks. We create a suite of 156 tests, more than three times the number conducted on HTTP/1.1, and analyze 12 popular proxies as well as 3 cloud proxies. This work not only investigates the current landscape of proxy implementations with respect to RFCs, but also examines the evolution of compliance over time in local proxies. We show that while there is an improvement compared to previous results in HTTP/1.1, none of the proxies are yet fully compliant.

CCS Concepts

• **Networks** → **Network measurement**; **Middleboxes / network appliances**; *Web protocol security*.

Keywords

HTTP, Protocol Compliance, Network Measurement

ACM Reference Format:
Mahmoud Attia, Ilies Benhabbour, and Marc Dacier. 2025. The Developer, the RFC, and the Middlebox: An HTTP/2 Compliance Story. In *Proceedings of the 2025 ACM Internet Measurement Conference (IMC '25), October 28–31, 2025, Madison, WI, USA*. ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3730567.3764447

1 Introduction

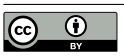
The Internet Engineering Task Force (IETF) defines protocol standards for the Internet known as Requests for Comments (RFCs). They define rules that govern machine interactions while allowing varying degrees of implementation flexibility. In theory, RFCs offer clear guidelines for developers. In practice, however, ambiguities within these standards persist, making it difficult to ensure compliance across implementations [5, 28]. The Hypertext Transfer Protocol (HTTP) is the backbone of web resource transfer. As of November 2024, Cloudflare reports that 80% of its routed traffic uses HTTP, with HTTP/2 accounting for 47.5%—equivalent to 38% of total Internet traffic [39, 40]. Given its substantial use, ensuring correct HTTP/2 implementation is crucial for maintaining efficiency and reliability in Internet communications.

Assessing a remote implementation's compliance to the standard is challenging due to intermediary devices, middleboxes, which interfere with supposedly end-to-end layers. While middleboxes enhance security, interoperability, and functionality beyond what endpoints can achieve, they also introduce risks. A recent survey [7] explores this duality, addressing issues such as new attack vectors [9], censorship [44], privacy concerns [47], intentional network degradation [17], and barriers to Internet protocol evolution [22]. In this work, to complement previous assessments of HTTP/2 server compliance [23, 41], we propose to investigate middlebox compliance with the HTTP/2 RFC from both the client and server perspectives.

This work builds on Rautenstrauch et al. [41] and Benhabbour et al. [5]. Rautenstrauch et al. extracted 106 rules from HTTP standards, testing them against nine web servers and nearly 10,000 websites. In contrast, Benhabbour et al. examined HTTP/1.1 compliance of proxies, analyzing how 12 popular ones handled 47 requirement-level rules defined in the RFCs. Although these studies provide valuable insights into the problem of HTTP compliance, their results are either incomplete or skewed. Rautenstrauch et al. [41] fail to assess the origin of compliance failures, and Benhabbour et al. [5] focus on the compliance of HTTP proxies with respect to HTTP/1.1 only.

This paper extends their work to study the more complex HTTP/2 protocol. This study represents, to our knowledge, the first focused effort to evaluate HTTP/2 compliance, with particular emphasis on proxy implementations. In line with previous results obtained on HTTP/1.1, our findings reveal significant inconsistencies in compliance enforcement. All our test descriptions and results are publicly available to the community. The key contributions of this paper are:

- We extract a list of 156 compliance tests from RFC9113 [45], covering requirements for clients, servers, and middleboxes.
- We investigate the impact of middleboxes on compliance with RFC9113 using up-to-date implementations of 12 popular proxies and 3 cloud proxies. This includes examining the compliance of proxies for HTTP/2 on both ends, as well as the effect of proxies that translate the HTTP/2 protocol into HTTP/1.1.
- We study 61 pairs of tests for end-to-end HTTP/2 (client-to-server and server-to-client) and demonstrate that requests and responses are handled differently by proxies, revealing an imbalance in compliance enforcement.



This work is licensed under a Creative Commons Attribution 4.0 International License.
IMC '25, Madison, WI, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1860-1/2025/10
https://doi.org/10.1145/3730567.3764447

- By comparing pairs of older and newer versions, we show that implementations tend to adopt stricter behaviors, preventing more non-compliant HTTP/2 messages from being forwarded to the servers.
- Building on the findings from previous studies and our own results, we propose an enhancement to future HTTP RFCs to improve clarity and coherence across implementations.

Section 2 reviews HTTP/2, its related literature, and the impact of network middleboxes on end-to-end communication. Section 3 outlines our methodology. Section 4 details the experimental setup, including the proxies studied and their configurations. Section 5 discusses the ethical considerations. Section 6 presents the results. Section 7 explores the findings and potential future work. Finally, Section 8 concludes the paper.

2 Background

HTTP is a protocol responsible for transferring data over the Internet. Originally developed by CERN between 1989 and 1991, it was created to facilitate efficient data sharing. Over the past three decades, HTTP has undergone significant evolution to improve performance and expand its supported features. To date, three major versions of the protocol exist: HTTP/1 (including its subversions HTTP/0.9, HTTP/1.0, and HTTP/1.1), HTTP/2, and HTTP/3. For the remainder of this paper, we focus on HTTP/2, as specified in RFC9113 [45]. This section begins with an overview of HTTP/2 (§2.1). Next, we motivate this work by discussing prior research on HTTP compliance (§2.2). Finally, we examine the role of HTTP middleboxes in HTTP compliance checking (§2.3).

2.1 HTTP/2 Overview

HTTP/2 is inspired by SPDY [3], a protocol developed by Google to enhance HTTP performance. One of the specific problems addressed by HTTP/2 is the head-of-line blocking issue in HTTP/1 [20]. In this subsection, we discuss the features that differentiate HTTP/2 from previous versions of HTTP. These features include binary framing, stream multiplexing, server push, and header compression. Binary framing and header compression are used to reduce bandwidth while the other features decrease the number of roundtrips overall, improving the efficiency of clients and servers communication. In the following paragraphs, we will describe each of these features in detail.

Binary framing. HTTP/2 uses binary framing to enhance the effectiveness of communication between the server and client. It involves dividing the stream of data into smaller, independent frames, each with a 12-byte header that contains the following fields: Length, Type, Flags, Reserved, and Stream Identifier. They are represented in Table 1 and detailed below.

- The 24-bit length field indicates the size of the data within the frame.
- The 8-bit type field specifies the structure and meaning of the frame.
- The 8-bit flags field communicates boolean flags that are specific to the type of frame.
- The 1-bit reserved field is always set to 0.
- The 31-bit stream identifier provides a unique identification number for the HTTP/2 stream.

Apart from the fields already mentioned, a frame also includes a payload whose size can vary depending on the length field. The payload immediately follows the stream identifier.

Bit	+0..7	+8..15	+16..23	+24..31
0	Length			Type
32	Flags			
40	R	Stream Identifier		
...	***Frame Payload***			

Table 1: Data is divided into smaller binary frames in HTTP/2 for more efficient transmission over the network

HTTP/2 uses a set of predefined frame types, including data, headers, priority, reset stream, and ping, among others. By using binary framing, HTTP/2 minimizes latency and optimizes the use of available resources, leading to faster, more efficient communication between clients and servers.

Stream multiplexing. HTTP/2 introduces the concept of stream, message, and frame to enable more efficient communication between clients and servers.

- A stream represents a bidirectional flow of bytes containing one or more messages, each with a unique identifier and optional priority information.
- A message is a complete sequence of frames that represents a logical request or response.
- Each message consists of one or more frames, the smallest communication unit in the protocol. A header frame, sent at the start of each message, carries the 31-bit stream identifier that uniquely identifies the HTTP/2 stream.

Multiplexing is an important feature in HTTP/2 that enables for multiple requests to be sent over a single TCP connection. By enabling multiple streams within a single connection, requests can be parallelized, resulting in faster content delivery and a better use of network resources. Figure 1 illustrates how multiplexing works for this version of the protocol.

Overall, the concepts of stream, message, and frame enable more efficient communication and improved performance, resulting in faster content delivery and a more responsive user experience.

Server push. HTTP/1.1 has a limitation that slows down the page rendering process. It forces the browser to download the HTML document first and then retrieve the necessary assets like CSS, JavaScript, and images. This results in multiple requests being sent from the client to the server, which can increase the overall load time of the website. However, with the introduction of HTTP/2, this issue has been resolved with the implementation of server push. Server push allows the server to preemptively push resources to the client before the client even requests them, enabling faster and more efficient loading of the webpage. For instance, a web server can push an external CSS style sheet immediately after the server responds to an *index.html* request, reducing the number of requests

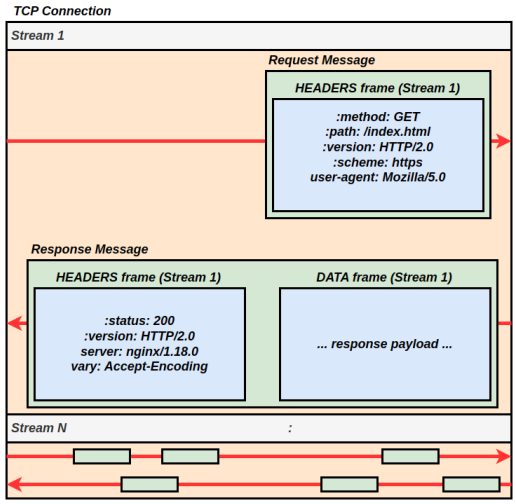


Figure 1: HTTP/2 stream exchange with binary framing for multiplexing

and speeding up the rendering of the webpage. This feature greatly improves the performance of the website and provides a better user experience.

Header compression. HTTP/2 introduces its own header compression algorithm called HPACK. The goal of HPACK is to reduce the overhead of the header fields that are sent in each request and response, and thus improve the performance of HTTP/2. HPACK is a hybrid compression algorithm that leverages a static dictionary of 61 commonly used header fields, some of which come with predefined values. In addition to a static dictionary, HPACK also uses a limited-size dynamic dictionary to store and compress headers encountered during a session. When new entries are added and the size limit is exceeded, the oldest entries are evicted in a first-in, first-out manner. Unlike algorithms such as DEFLATE, which use adaptive Huffman encoding, HPACK employs static Huffman encoding. The codebook for static Huffman encoding is precomputed based on the frequency of symbols. HPACK was specifically designed to be resilient to attacks that exploit the adaptive nature of dynamic Huffman encoding, such as the Compression Ratio Infoleak Made Easy (CRIME) attack [38]. In such attacks, an adversary with control over part of a header can infer secret information—like authentication cookies—by observing changes in payload size [31].

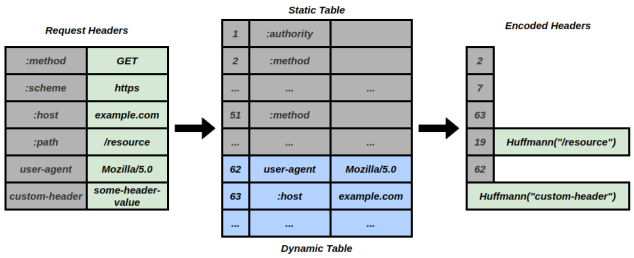


Figure 2: HPACK header compression processing

2.2 HTTP Compliance

RFCs define protocols and outline implementation requirements using terms such as MUST, SHOULD, and MAY. In practice, however, web server implementations often deviate from these prescribed standards. In 2001, Krishnamurthy et al. introduced PRO-COW [32], a framework for assessing HTTP/1.1 compliance, and found that many popular web servers failed to implement key features such as persistent connections and support for new HTTP methods. That same year, The Measurement Factory (TMF) evaluated HTTP compliance in web intermediaries, identifying numerous standard violations in Squid HTTP proxies [48].

Subsequent studies continued to highlight widespread non-compliance. In 2008, Adamczyk et al. [1] analyzed 176 popular websites for support of methods like OPTIONS, TRACE, GET, conditional GET, HEAD, and CONNECT, uncovering frequent misconfigurations and implementation flaws. A decade later, Nguyen et al. [37] introduced a tool with 397 test cases to evaluate web caching servers' adherence to RFC 7234 [15]. In a follow-up study [36], they used this tool to test seven popular caching servers, including Nginx, Apache, and Squid. They found that some of these servers incorrectly cached responses to requests containing Authorization headers, violating the standard and potentially exposing sensitive data. Also, Rautenstrauch et al. [41] examined HTTP rule violations across nine major web server implementations and nearly 10,000 websites, finding that the vast majority violated at least one of the 106 rules tested. However, the tests specifically targeting HTTP/2 were limited to just five. Benhabbour et al. [5] investigated HTTP/1.1 compliance in the presence of middleboxes, demonstrating that the behavior of middleboxes complicates the assessment of remote endpoint compliance, as HTTP/1.1 requests and responses can be altered along the network path.

Specialized tools have also emerged for newer protocol versions. For example, h2spec [23] targets HTTP/2, verifying compliance with features like header compression (HPACK). To the best of our knowledge, our study is the first to examine HTTP/2 compliance, specifically targeting HTTP/2 proxy implementations.

2.3 HTTP Middleboxes

Network middleboxes, as described in RFC 3234 [11], are intermediary devices that disrupt the end-to-end principle established by Saltzer et al. [42]. This principle asserts that network functions should be *"correctly implemented only with the knowledge and help of the application standing at the endpoint"*. The emergence of middleboxes has effectively transformed the Internet's structure from an end-to-end model to an end-to-middle-to-end framework. While middleboxes can improve network performance and security, they also introduce new vulnerabilities [7].

An example of such vulnerabilities is HTTP Request Smuggling (HRS), first identified in 2005 [21]. This attack exploits differences in how web proxy middleboxes and end servers interpret the same protocol, enabling attackers to circumvent access controls imposed by proxies, CDNs, and firewalls, allowing unauthorized requests to be sent. Over a decade later, Chen et al. [12] unveiled an attack called *"Host of Troubles"*, which took advantage of ambiguities in Host header implementations. Specifically, when multiple Host headers were included in a single request, inconsistencies in

their handling across implementations created opportunities for exploitation.

In 2019, James Kettle showed that inconsistent interpretations of the Content-Length and Transfer-Encoding headers—between middleboxes and end servers—could enable session hijacking and credential theft [28]. Subsequent works, such as T-Reqs [26], HDiff [43], HTTP Garden [27], and Gudifu [24], have applied differential fuzzing techniques to uncover similar flaws in HTTP implementations. In 2021, Kettle further revealed that downgrading from HTTP/2 to HTTP/1 could also be exploited for desynchronization attacks, emphasizing a gap in server-side support for HTTP/2 despite its growing client adoption [29]. The following year, Jabiyev et al. [25] introduced Frameshifter, a tool designed to detect anomalies in HTTP/2 to HTTP/1.1 downgrades, trace their origins, and assess potential vulnerabilities.

Web Cache Deception (WCD) is a type of attack that exploits differences in how frontend middleboxes and backend servers interpret requests. It was first introduced by Omer Gil in 2017 [18]. WCD leverages caching proxies to improperly store sensitive user data. In a 2020 study, Mirheidari et al. [33] revealed that 295 of the Alexa Top 5K websites leaked sensitive authentication and authorization tokens. A follow-up study by the same authors in 2022 [34] found that 1,188 sites in the Alexa Top 10K remained vulnerable to WCD attacks.

The vast majority of these attacks are caused by non compliant behaviors of middleboxes. Our goal in this study is not to reproduce any of these known attacks or find new vulnerabilities but, rather, to study systematically the amount of non compliance in middleboxes.

3 Methodology

This section outlines our methodology for evaluating network middlebox compliance with the HTTP/2 standard. Subsection 3.1 defines the properties under study, specifying in-scope and out-of-scope aspects. Subsection 3.2 details the classification of proxy behaviors. Subsection 3.3 outlines our definition of compliance, based on the HTTP/2 standard [45]. Subsection 3.4 presents briefly the measurement framework used to perform compliance campaigns.

3.1 Scope

In this paper, we do not test end-server implementations. Instead, we focus on HTTP proxy implementations—a class of middleboxes that HTTP payloads may encounter along the network path. Building on Benhabbour et al. [5], we extend their work to cover HTTP/2. Our research addresses the following questions:

- **Q1:** Do HTTP/2 proxies accept or reject non-compliant HTTP/2 messages from clients, either as-is or with modifications?
- **Q2:** Do HTTP/2 proxies accept or reject non-compliant HTTP/2 messages from servers, either as-is or with modifications?

RFCs define protocols and implementation rules using terms like MUST, SHOULD, and MAY. Specifically, RFC 9113 [45] governs HTTP/2. This paper focuses on requirement-level rules—specifically, those defined by the RFC using the MUST keyword, which, according to RFC 2119 [10], indicates "absolute requirements" and allows no freedom in implementation. By contrast, MAY rules are "truly optional" and excluded from this study, while SHOULD rules are recommendations. Although SHOULD rules (22 in total) are potentially relevant, they are generally too broad or context-dependent to verify (e.g., "Servers SHOULD use other contextual information in determining priority of requests in the absence of any priority signals"). We identified 223 MUST requirement rules, but excluded some from our study due to inapplicability or lack of direct relevance. For example, Section 9 defines 14 rules related to TLS rather than HTTP, while Section 10 includes 8 rules that are difficult to implement or too subjective (e.g., "Compression MUST NOT be used if the source of data cannot be reliably determined"). Among the applicable rules, 10 are direct duplicates (e.g., "Trailers MUST NOT include pseudo-header fields" and "Pseudo-header fields MUST NOT appear in a trailer section"). We focused on tests related to frame content analysis. In total, we manually derived 156 concrete test cases from RFC 9113, in some instances splitting a single rule into distinct client and server tests and manually crafting the probes. This represents a substantially larger corpus than Benhabbour's 47 tests on HTTP/1.1 intermediaries [5] and Rautensrauch's 5 tests on HTTP/2. The complete set of in-scope cases is listed in Appendix B. To encourage reproducibility and extension, we release these test cases to the community; they are reusable within the measurement framework presented later in Subsection 3.4 and are available on Github at https://github.com/nopasaran-org/nopasaran-tests. A mapping of overlapping HTTP/1.1 tests from Benhabbour et al. [5] to our HTTP/2 tests is provided in Appendix A.

3.2 Behavior Classification

In our experiments, we control both endpoints of an HTTP connection and study the compliance of an intermediate network element. To assess this compliance, we send compliant as well as non-compliant HTTP/2 messages from the client to the server and vice versa. For HTTP/1.1, Benhabbour et al. [5] have shown that proxies can reject, accept as-is, or modify non-compliant requests and responses. This paper enriches and expands the scope of their work by classifying proxy behavior in HTTP/2 messages according to the taxonomy presented in Table 2.

Rejected	
(D) Dropped	The proxy discards the message
(E) Error	The proxy returns a 500 Status Code Error to the sender
(G) Goaway	The proxy sends a GOAWAY frame to signal a serious error and to initiate the connection shutdown
(R) Reset	The proxy sends an RST_STREAM frame to terminate a stream without impacting other streams
Accepted	
(U) Unmodified	The message is forwarded unchanged
(M) Modified	The message is altered before delivery (e.g., for content sanitization)

Table 2: Non-Compliant Message Handling Outcomes

It is important to note that some proxies downgrade HTTP/2 connections from client to proxy into HTTP/1.1 connections from proxy to server. In such cases, a non-compliant HTTP/2 message is considered accepted (rather than accepted modified or accepted unmodified) if its reception by the proxy results in a HTTP/1.1 message being sent to the server.

3.3 Compliance Definition

The following is the compliance definition used in our study, along with an explanation based on RFC 9113:

Definition

A proxy is considered compliant if it reports an error upon encountering a non-compliant HTTP/2 message. This can be done by sending a 500 Status Code Error (E), a GOAWAY frame (G), or a RST_STREAM frame (R).

According to RFC 9113 [45], HTTP/2 defines two types of errors: **connection errors**, which invalidate the entire connection, and **stream errors**, which affect only a single stream. For connection errors, RFC 9113 states:

> "If a frame causes a connection error, that error MUST be reported. [...] An endpoint that encounters a connection error SHOULD first send a GOAWAY frame. [...] After sending the GOAWAY frame for an error condition, the endpoint MUST close the TCP connection."

Discarding a frame without reporting an error is therefore non-compliant. While sending a GOAWAY frame is recommended (SHOULD), it is not strictly required. We thus consider that sending a 500 Error message is also seen as a compliant way by the RFC authors to report an error. For stream errors, RFC 9113 notes:

> "[...] an endpoint MAY choose to treat a stream error as a connection error. [...] An endpoint that detects a stream error sends a RST_STREAM frame (Section 6.4) that contains the stream identifier of the stream where the error occurred."

Here, RFC 9113 allows an endpoint to treat a stream error as a connection error, meaning it may send a GOAWAY frame—though this is not mandatory. However, if the error is treated as a connection error, the endpoint MUST report it. The specification also notes that an endpoint sends a RST_STREAM frame upon detecting a stream error, but the absence of normative keywords such as MUST or SHOULD implies that sending the reset frame is not strictly required. For stream errors, we accept either a GOAWAY, a RESET_STREAM or a 500 Error to be compliant ways to deal with an error. Finally, Section 8.1.1 of RFC 9113 specifies that intermediaries MUST NOT forward malformed requests or responses. Such messages must be treated as stream errors.

To summarize, the only mandatory behavior required by RFC 9113 is that non-compliant messages MUST NOT be forwarded and that the error MUST be reported. This requirement can be satisfied by sending an error status code (E), a GOAWAY frame (G), or a RST_STREAM frame (R).

3.4 Measurement Framework

To conduct our HTTP/2 measurements, we leveraged the measurement framework NoPASARAN, which was proposed and used by Benhabbour et al. in their HTTP/1.1 compliance studies [5, 6]. NoPASARAN is an open source distributed middlebox measurement platform where nodes execute assigned tasks, tests leverage two communication channels, and orchestration follows a hierarchical design. We enriched this framework to support our HTTP/2 measurement campaigns. Figure 3 shows the framework, and the following summarizes its structure:

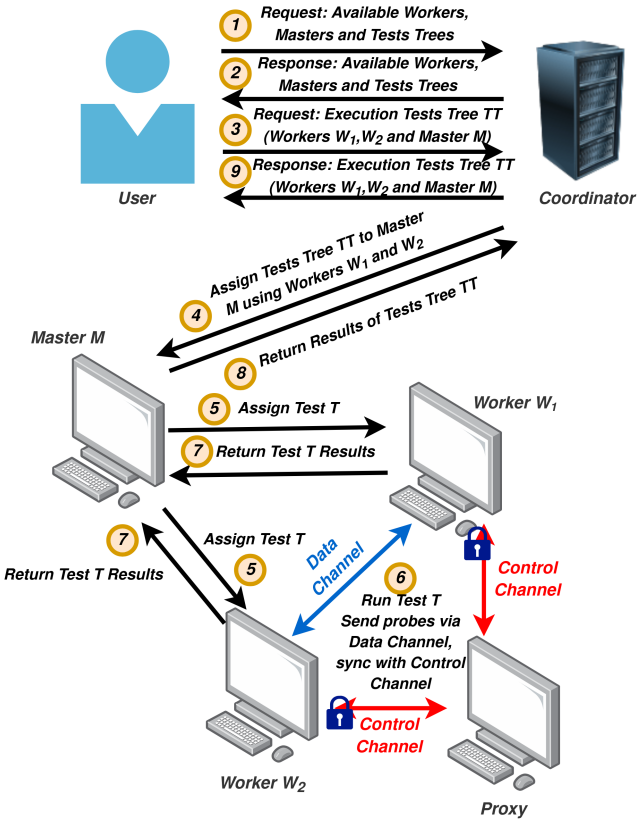


Figure 3: Process flow of the NoPASARAN framework. The user interacts with the coordinator, which delegates orchestration to a master. The master manages tests executed by worker pairs, optionally using a proxy.

- **Node Roles:** Nodes are assigned one of four roles: coordinator, master, worker, or proxy. The coordinator interfaces with the end user to manage measurement campaigns. It assigns a master to oversee each test. Workers perform the active probing between node pairs, measuring what is sent and received. A proxy is used when direct communication between workers is not possible.
- **Data & Control Channels:** Workers send probes over a data channel and coordinate the test flow via a mutually authenticated and end-to-end encrypted control channel.
- **Testing Hierarchy:** The coordinator manages measurement campaigns, each defined as a decision tree, where each tree node represents a test. Tests are modeled as state machines, with instructions executed sequentially by workers.

We modified the original code of H2, a Python library [4], by overloading functions to generate non-compliant HTTP/2 messages. These changes, intended for future HTTP/2 compliance testing, were merged with the consent of the NoPASARAN maintainers and made

publicly available to the community, to be used or enriched further by other researchers.

4 Experimental Setup

The tested proxy servers, their respective versions and release dates are listed in Table 3

We evaluated 12 local proxy implementations and 3 cloud-based proxies. The local implementations were selected based on their open-source availability and popularity, measured primarily by GitHub star counts. We assume that a higher star count reflects greater adoption, coverage, and representativeness in practice. Unlike Benhabbour et al. [5], we consider the version and release date of the tested local proxies, as these factors may affect our results. All tested implementations are production-grade HTTP/2 proxies, except Mitmproxy, which targets "privacy measurements and penetration testing" [35]. We include it to verify behavioral differences relative to proxies with different design goals. Only a subset of proxies support end-to-end HTTP/2 (both downstream and upstream, referred to as H2EE). To expand our analysis, we also test HTTP/1.1 downgrade proxies, where the client-to-proxy connection uses HTTP/2, but the proxy-to-server connection only supports HTTP/1.1 (H2H1). Additionally, some proxies require TLS support, as RFC 9113 prohibits H2C. Our tests accommodate TLS, applying it when necessary. Kondracki et al. [30] examined phishing toolkits such as Evilginx [19], Muraena [46], and Modlishka [14]. However, our experiments have shown that these tools do not support HTTP/2, and therefore cannot be measured or compared with other implementations in this study. Implementations were built on an Ubuntu 24.10 virtual machine (VM), while HTTP/2 probes were sent from two additional worker VMs on the same hypervisor with direct TCP connections, to the proxy as well to each other, to prevent any possible external interference.

Local Proxies	Version	Release Date	H2EE	H2H1	TLS	⋆
Apache	2.4.63	2025-01-23	✓	✓		3.8k
Caddy	2.9.1	2025-01-08	✓			66.6k
Traefik	3.3.5	2025-03-31	✓	✓		56.5k
Envoy	1.34.1	2025-05-07	✓	✓		26.5k
H2O	26b116e95	2025-01-30[1]	✓	✓	✓	11.2k
HAProxy	3.2.0	2025-05-09	✓	✓		5.9k
Mitmproxy	11.1.0	2025-01-12	✓		✓	40.4k
Nghttpx	1.62.1	2024-05-19	✓	✓		4.8k
Node	20.16.0	2024-07-24	✓	✓		113k
Varnish	7.7.0	2025-03-17		✓		3.9k
Nginx	1.28.0	2025-04-23		✓	✓	27.9k
Lighttpd	1.4.76	2024-04-12		✓	✓	664
Cloud Proxies	**Version**	**Release Date**	**H2EE**	**H2H1**	**TLS**	**⋆**
Cloudflare	N/A	N/A	✓	✓	✓	N/A
Azure Gateway	N/A	N/A		✓	✓	N/A
Fastly	N/A	N/A		✓	✓	N/A

Table 3: Tested proxies, along with their versions, release dates, Github stars (⋆), and whether they have been tested with H2EE, H2H1, and require TLS. H2EE indicates full HTTP/2 communication between the client and server, while H2H1 represents HTTP/2 to HTTP/1.1 translation.

In this paper, we do not investigate geolocation discrepancies of CDN providers when testing cloud proxies, leaving this aspect for future work. In these experiments, we use two workers hosted on Virtual Private Servers (VPS) provided by Linode, both belonging to Akamai Technologies Inc. (ASN 63949). One worker acts as the client, sending non-compliant requests to the cloud proxy, while the other functions as the server, returning non-compliant responses. Table 4 summarizes the role, IP address, and location of both workers.

Role	IP Address	Location
Client	172.104.229.16	Frankfurt, Germany
Server	172.236.62.68	Melbourne, Australia

Table 4: Experimental Setup for Cloud Proxy Testing

Experiments on cloud proxies were conducted on May 7, 2025. To ensure that they were not affected by intermediate transparent proxies at the network edges (e.g., any default one from Linode), we first perform a direct TCP handshake between two Linode-hosted machines. Transparent proxies that intercept and split TCP connections typically modify sequence and acknowledgment numbers. By verifying that these numbers match at both endpoints on port 443—as all cloud proxies in our study require HTTP/2 over TLS—we confirm the absence of any transparent proxy in the vicinity of the endpoints.

5 Ethics

Measurements on self-hosted proxies are performed in an isolated environment. For the three cloud providers used in this study, we limit measurements by maintaining a minimum interval of one day between campaigns on a single cloud. Additionally, we carefully craft payloads to avoid known vulnerabilities, such as HTTP request smuggling. All measurements conducted from Linode-hosted machines comply with their Acceptable Use Policy [13]. The results of this survey were responsibly disclosed to the affected parties to help enhance their security posture and mitigate potential risks. Since the observed behaviors did not, according to the implementers, imply a direct security violation (in the absence of a proof-of-concept exploit) nor an interoperability concern, discussions generally did not progress beyond initial exchanges. Nonetheless, the vendors engaged with our reports in a constructive manner, and their feedback was valuable in refining our methodology and ensuring the accuracy of our findings.

6 Evaluation

This section presents the results of our measurement campaigns[2]. Subsection 6.1 provides a high-level overview, highlighting differences between various proxies and lessons that can be learned by comparing these results to those obtained in [5] for HTTP/1.1. Subsection 6.2 delves deeper, using specific examples to illustrate the results more concretely. Subsection 6.3 examines the differences in how proxies handle HTTP/2 packets on the client and server sides. Subsection 6.4 analyzes the evolution of proxy compliance over time. Finally, Subsection 6.5 compares the behaviors of the same proxies when configured for H2EE versus H2H1.

[1] It is worth noting that H2O has been in version 2.3.0 since 2019-08-13. Despite its early release date, it continues to receive regular updates. We used the proxy version from the commit identifier 26b116e95, dated 2025-01-30.

[2] All test descriptions and results are publicly available at https://github.com/attia-mahmoud/HTTP2-Compliance-Tests.

	Rejected					Accepted		
Local Proxy H2EE	**D**	**E**	**G**	**R**	**%**	**U**	**M**	**%**
Apache	48	40	67	0	99.36%	1	0	0.64%
Caddy	50	65	29	0	92.31%	6	6	7.69%
Traefik	127	0	29	0	100%	0	0	0%
Envoy	52	31	63	4	96.15%	4	2	3.85%
H2O	89	7	59	0	99.36%	0	1	0.64%
HAproxy	40	24	81	1	93.59%	5	5	6.41%
Mitmproxy	21	0	0	4	16.03%	131	0	83.97%
Nghttpx	55	0	61	34	96.15%	5	1	3.85%
Node	70	14	61	0	92.95%	3	8	7.05%
Cloud Proxy H2EE								
Cloudflare	73	63	6	5	94.23%	2	7	5.77%
Subtotal	**625**	**244**	**456**	**48**	**88.01%**	**157**	**30**	**11.99%**
Local Proxy H2H1								
Varnish	36	0	31	0	85.90%	11		14.10%
Nginx	49	0	27	0	97.44%	2		2.56%
Lighttpd	44	0	30	1	96.15%	3		3.85%
Cloud Proxy H2H1								
Azure-AG	76	0	0	0	97.44%	2		2.56%
Fastly	26	0	43	9	100%	0		0%
Subtotal	**231**	**0**	**131**	**10**	**95.38%**	**18**		**4.62%**
Total	**1745**				**89.49%**	**205**		**10.51%**

Table 5: Proxy test results by category when handling non-compliant HTTP/2 messages. Categories include: Dropped (D), Error (E), Goaway (G), Reset (R), Accepted and forwarded unmodified (U), and Accepted and forwarded with modifications (M).

6.1 Global Trends in HTTP/2 Compliance Among Proxies

Proxies exhibit different behaviors when processing payloads that do not comply with specifications. Table 5 summarizes the aggregated results of 1950 unique test cases across all proxies: 1560 from 10 H2EE proxies (156 each) and 390 from 5 H2H1 proxies (78 each). For proxies that do not require the mandatory usage of TLS with HTTP/2, we evaluated both plaintext and TLS modes and found no differences, so the results are reported only once. This outcome is consistent with the layering principle: application data is passed to lower layers, including TLS, which processes it without altering semantics.

A key observation is that most proxies reject non compliant payloads, suggesting a general resistance to potential attacks like HTTP request smuggling. However, by simply dropping non-compliant messages in 856 cases (625 for H2EE proxies and 231 for H2H1 proxies) they still violate the RFC9113 specification since no notification of the error is done (as explained in Section 3. In 205 cases (157 unmodified and 30 modified for H2EE proxies, and 18 accepted for H2H1 proxies), the payload was accepted and forwarded—131 of which were from Mitmproxy, which forwarded the message unchanged in all instances. As expected, Mitmproxy prioritizes transparency by forwarding most non-compliant messages unmodified, favoring stealthiness over strict protocol adherence—corroborating our findings. Excluding Mitmproxy, 74 cases involved non-compliant payloads being forwarded to the final recipient which is worrisome and definitely forbidden by RFC9113 in its Section 8.1.1. Compared to earlier results in HTTP/1.1 [5], which showed that out of a total of 564 tests (12 proxies × 47 test cases), 128 test cases (22.7%) were rejected, HTTP/2 implementations tend to reject non-compliant packets significantly more often. This behavior may stem from the way RFC9113 is written. As mentioned before, it explicitly states that malformed requests or responses MUST NOT be forwarded. In contrast, RFC9110 [16], which defines HTTP semantics, indicates that all HTTP requirements applicable to an origin server also apply to the client-side of a gateway or reverse proxy—but it does not explicitly impose a strict prohibition on forwarding malformed messages for intermediaries. This leaves greater flexibility to implementers.

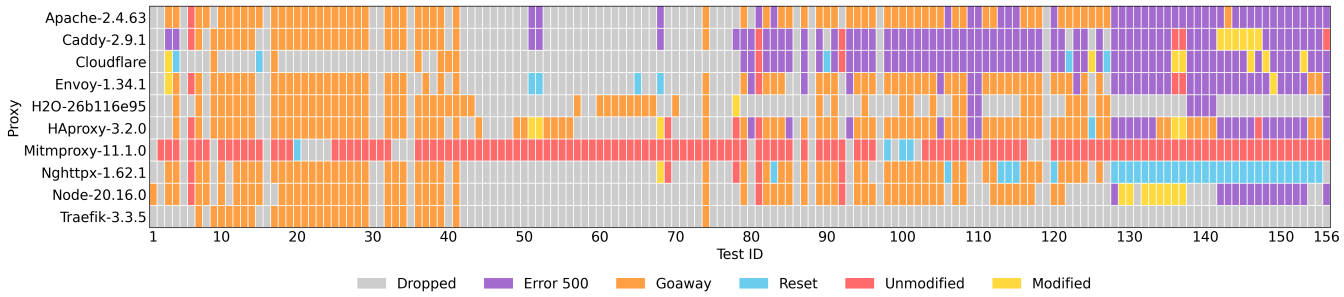
6.2 In-Depth Analysis of HTTP/2 Compliance

The previous subsection provides insight into the overall trend, showing not only that proxies fail to fully comply with RFC9113, but also that they interpret and implement requirement-level rules differently. In this subsection, we dig deeper and present concrete examples of non-compliant behaviors.

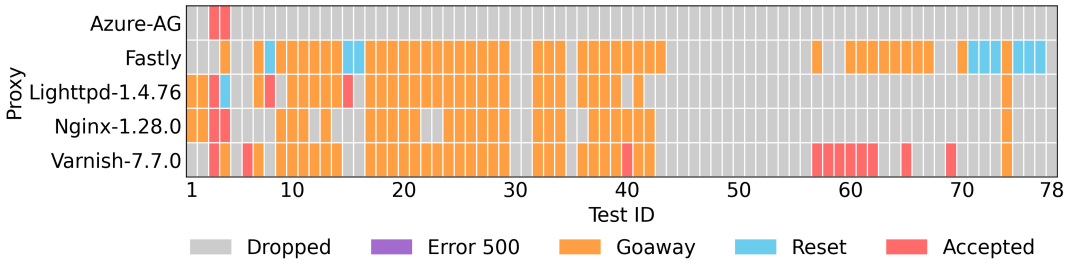
Figure 4 shows the vectorized behaviors observed for each test case by both HTTP/2 end-to-end proxies and HTTP/2 to HTTP/1.1 downgrading proxies. Out of the 156 test cases (excluding Mitmproxy), 39 revealed instances where a proxy accepted the payload—either by forwarding it unmodified or after modifying it. For example, the standard specifies that "Pseudo-header fields MUST NOT appear in a trailer section." In our test, we added the :path pseudo-header to a trailer. Instead of reporting an error, Cloudflare modified the payload before forwarding it by removing the non-compliant pseudo-header. While some proxies sanitize the data before forwarding, other behaviors may introduce security risks. RFC9113 states that "A field value MUST NOT start or end with an ASCII whitespace character." We split this requirement into two test cases—one for a leading whitespace and one for a trailing whitespace. In both cases, Cloudflare, HAProxy, and Node accepted the payload and forwarded it to the final recipient after sanitizing it (modified). Caddy and Envoy, on the other hand, accepted the payload and forwarded it as-is (unmodified). This is worrying because James Kettle demonstrated that whitespace in the :method pseudo-header can lead to request line injection [29].

Non-compliance can occur at any stage of communication between an endpoint and a proxy—for example, during connection initialization: RFC9113 states that the client preface "MUST be followed by a SETTINGS frame". Upon testing, we see that Cloudflare and Envoy accept the connection despite the absence of a SETTINGS frame in the client's preface. Non-compliance can also originate from a violation of the HTTP/2 state machine: RFC9113 states that "If a DATA frame is received whose stream is not in the 'open' or 'half-closed (local)' state, the recipient MUST respond with a stream error (Section 5.4.2) of type STREAM_CLOSE". When tested in the half-closed (remote) state, four of our proxies—Caddy, Cloudflare, Nghttpx, and Node—accepted the DATA frame despite the stream not being in one of the permitted states.

The results seem to indicate that we can extract a fingerprint of a proxy implementation from the vectorized representation of behaviors using a subset of tests, based on the types of error messages it produces—or the absence of such messages. However, since the versions and platforms on which the proxies are built may affect the



(a) H2EE proxies behavior by test ID



(b) H2H1 proxies behavior by test ID

Figure 4: Behaviors exhibited by the different HTTP/2 proxies across tests, identified by their test IDs (see Appendix B).

results, we leave a thorough investigation of this aspect to future work.

6.3 Client and Server Side Discrepancies in HTTP/2 Compliance

Benhabbour et al. examine how proxies handle HTTP/1.1 requests and responses, highlighting inconsistent behavior in applying similar rules—such as checking for carriage-return or line-feed characters in field values [5]. However, their analysis lacks statistical balance, with only 5 tests on requests compared to 42 on responses. In this subsection, we focus on this issue within HTTP/2 proxy implementations. We present a balanced set of 78 requirement-level tests for client-to-server messages and 78 for server-to-client messages. Of these, 17 tests apply only to the client and another 17 only to the server. The remaining 122 tests, derived from the same 61 RFC requirements that can be applied in both directions, bring the total to 156 tests ($61 \times 2 + 17 + 17$).

Figure 5 illustrates the non-compliance of the ten HTTP/2 end-to-end proxies. The results show that for a set of the same given rules, proxies handle non-compliant client- and server-side messages differently, which may suggest redundant code paths in their implementations that could otherwise be reused. Most implementations exhibit better compliance in the server-to-proxy direction, with the exception of H2O and Traefik. While this demonstrates discrepancies between client- and server-side compliance with requirement-level rules, the comparison includes rules that apply specifically to one role (e.g., the client MUST, the server MUST). However, some rules apply regardless of whether the endpoint is a client or server. Figure 6 illustrates how HTTP/2 end-to-end proxies

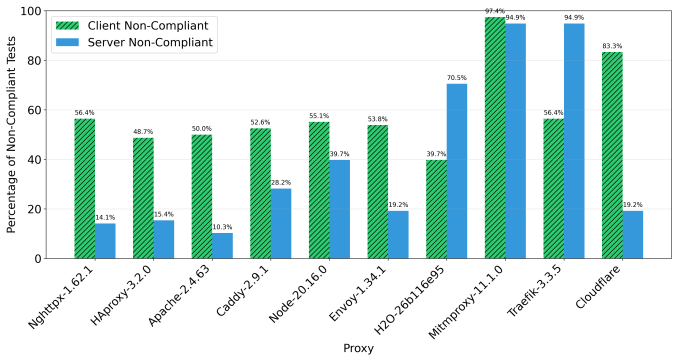


Figure 5: Non-compliance ratio per proxy for HTTP/2 end-to-end: striped green represents client-to-proxy, solid blue represents server-to-proxy. Despite balanced test coverage, compliance is uneven between client and server directions.

behave differently on the client and server sides for the same set of requirement rules—specifically, the 61 rules that apply to both sides simultaneously, rather than to only one. All ten proxies exhibited discrepancies, with nine showing at least a 40% discrepancy rate—Mitmproxy being the only exception. This again suggests that proxy implementations may not employ reusable components, resulting in inconsistent enforcement of the same requirement rules on the client and server sides.

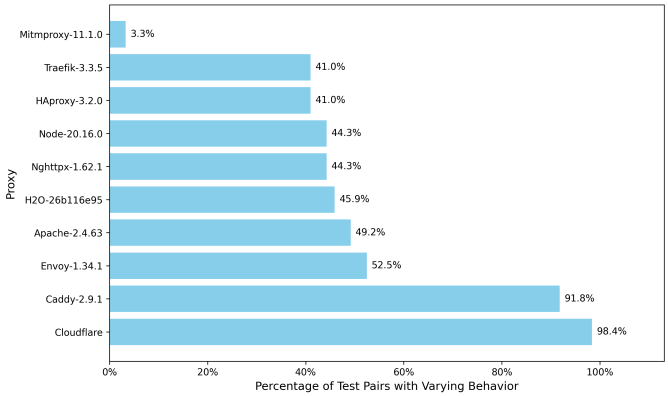


Figure 6: Percentage of test cases (from 61 client–server test pairs) that exhibited different behaviors for each proxy. Higher percentages indicate greater discrepancies between client- and server-side behavior.

6.4 Evolution of HTTP/2 Compliance in Proxies

RFC 9113, published in June 2022, obsoletes RFC 7540 from 2015 [2]. In the following paragraphs, we examine whether proxy implementations have adapted in response to the updated specifications. Our focus is not on whether earlier implementations were expected to comply with RFC 9113 prior to its publication, nor on assessing compliance with aspects of RFC 7540 outside the scope of this paper. Instead, we aim to investigate how implementation behavior may have evolved since the release of RFC 9113. Since RFCs take a long time to be published—and drafts are already publicly available before official publication—implementers can begin to take new recommendations into account before the official release of the revised RFC. Therefore, instead of considering the latest versions preceding that official release date, we deployed the latest versions of the ten local proxies listed in Table 6, using versions preceding the very first draft of RFC 9113, released in August 2020. This minimizes the risk to see these versions already taking the new recommendations into account since the early draft was not available at that time. Mitmproxy and Caddy are excluded from our analysis. Mitmproxy is omitted because, unlike the production-grade proxies, findings from the previous subsections show that its primary focus is not compliance. In contrast, this older version of Caddy behaves differently from the other proxy implementations. While the others wait to receive client frames before forwarding them, this Caddy version preemptively sends HTTP/2 frames to the server upon connection. Our NoPASARAN-based scenario is not designed for this specific corner case, preventing it from functioning correctly.

Figure 7 shows the behavioral changes observed across the ten local proxies, between the versions listed in Table 6 and those in Table 3. Each line represents the changes between two versions of the same proxy (before and after the publication of the first draft of RFC 9113). The x-axis shows the observed behaviors. For each behavior and each pair of proxy versions, we calculate the difference in the number of occurrences of that behavior between the older and newer version. A positive value for a given behavior indicates that it is more observed with the newer than with the older version.

Local Proxies	Version	Release Date
Apache	2.4.43	2020-03-26
Traefik	2.2.8	2020-07-28
Envoy	1.15.0	2020-07-07
H2O	2.3.0@0a9ddbd	2020-07-30
HAProxy	2.2.0	2020-07-07
Nghttpx	1.41.0	2020-06-02
Node	14.7.0	2020-07-29
Varnish	6.4.0	2020-03-16
Nginx	1.18.0	2020-04-21
Lighttpd	1.4.55	2020-01-31

Table 6: Version numbers and release dates of the older local implementations tested.

A notable change is in GOAWAY handling. Three proxies—Envoy, H2O, and Lighttpd—use it more frequently in recent versions, with increases in 31, 16, and 30 test cases respectively. Other implementations show minimal change (fewer than four cases). This likely reflects the fact that RFC 7540 already recommends sending GOAWAY on connection errors, so most earlier implementations had already adopted this behavior. Another shift is in "dropping" behavior. Most proxies now drop messages less frequently: Lighttpd, HAProxy, H2O, Varnish, Envoy, and Node reduced this behavior in 34, 29, 24, 12, 11, and 6 test cases respectively. By contrast, the latest version of Traefik drops messages more frequently (12 cases). In earlier versions, it would send reset frames (9 cases) or error messages (2 cases) instead of silently discarding them—indicating a shift from a more compliant implementation to a less compliant one under RFC 9113. Overall, proxies are increasingly restricting raw forwarding of probes, with the exception of Node, which forwarded one additional probe as-is. Several implementations also sanitize non-compliant messages before forwarding; for example, Node modified and forwarded 7 such cases. While not immediately harmful to security or interoperability, this diverges from RFC 9113. More concerning is Varnish, whose newer version accepted 11 more invalid probes than before, a clear compliance regression. Finally, some proxies show little evolution. For example, Apache only shifted a handful of cases: 3 GOAWAY behaviors and 1 as-is forward were replaced by 2 drops and 2 error reports.

Overall, many proxy implementations have evolved since the first draft of RFC 9113, improving compliance in areas like GOAWAY handling and reducing silent drops. However, changes are uneven, with some regressions and divergences remaining, highlighting ongoing inconsistencies across implementations.

6.5 HTTP/2 End-to-end vs HTTP/2 to HTTP/1.1 Downgrade

This section highlights the behavioral differences observed in proxies that support both HTTP/2 end-to-end and HTTP/2 to HTTP/1.1 downgrade modes. Among the proxies tested with HTTP/2 end-to-end, eight were also evaluated in downgrade mode (see Table 3). Caddy was excluded from these tests due to the compatibility issues explained before. The test cases used for comparison were limited to those involving client-side behavior.

Figure 8 illustrates that the same proxy implementation can exhibit different behaviors depending on whether it is configured

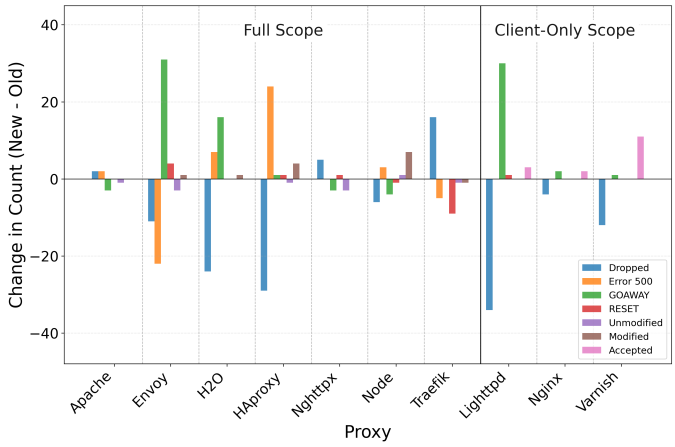


Figure 7: Change in behavior by proxy pair for each behavior type. Positive values indicate increased adoption of a behavior by a proxy; negative values indicate the behavior was reduced or removed through implementation changes.

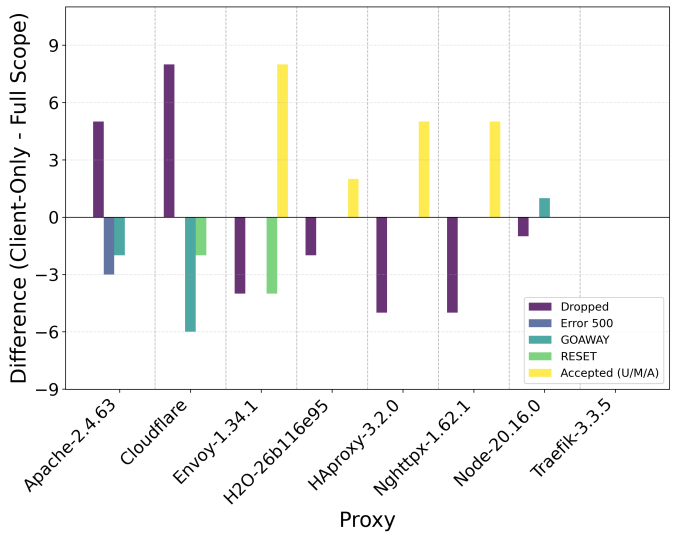


Figure 8: Behavioral differences between proxy pairs (H2EE vs. H2H1) across behavior types. Positive values indicate that proxies configured with H2H1 exhibit more of a given behavior than their H2EE counterparts; negative values indicate the reverse.

for H2H1 or H2EE. In this comparison, accepted messages in H2H1 mode are compared with both modified and unmodified accepted messages in H2EE, since both configurations involve forwarding messages.

A key observation is that only Traefik behaves consistently across the two configurations; all other proxies show differences. Another notable point is that Cloudflare, Apache, and Envoy—when configured for H2H1 translation—signal fewer errors, even though RFC 9113 specifies that they should. For example, Cloudflare drops 8 messages instead of sending 6 GOAWAY frames and 2 Reset frames. A further finding is that proxies configured for H2H1 tend to forward more non-compliant messages. Specifically, Envoy, HAProxy, Nghttpx, and H2O accept an additional 8, 5, 5, and 2 non-compliant messages, respectively. For instance, H2O permits a connection to continue without the client sending a SETTINGS frame or connection preface in HTTP/2–to–HTTP/1.1 mode, but correctly rejects such connections in HTTP/2 end-to-end mode. Despite these behavioral differences, some test cases reveal consistent non-compliant behavior across both modes. For instance, Envoy forwards HEADERS frames that contain field values starting or ending with ASCII whitespace characters in both HTTP/2 end-to-end and downgrade modes, even though RFC 9113 explicitly forbids such characters in header fields.

Overall, the comparison shows that the same proxy implementation can behave differently depending on whether it is configured for H2EE or H2H1 downgrade mode. Most implementations are less compliant in downgrade mode, forwarding more non-compliant messages and signaling fewer errors, though some behaviors remain consistently non-compliant across both modes.

7 Discussion & Future Work

This section presents a recommendation based on the observations discussed in Subsection 7.1 and outlines future work in Subsection 7.2.

7.1 Recommendation

The stricter design of HTTP/2, which reduces implementation flexibility, is generally preferable to the looser constraints of HTTP/1.1, which offer developers more freedom. However, despite that, our results indicate that HTTP/2 implementations are not always fully compliant with the specification.

Although our study does not identify new vulnerabilities, the question of implementation flexibility remains relevant. While some may view such flexibility as advantageous, previous attacks exploiting inconsistencies in message interpretation indicate that it can also introduce risks. From this perspective, limiting freedom in protocol design can contribute to stronger security and improved interoperability.

We recommend that each new RFCs be systematically provided with a formal compliance test suite. This suite would act as a normative reference, specifying expected behaviors for both clients and servers when dealing with non-compliant messages. For instance, given a HTTP/2 message containing leading whitespace in a field value, the correct behavior MUST be to send a GOAWAY or RST_STREAM frame. Such an explicit mapping of inputs to expected outcomes provides much clearer guidance than the current specification, which often leaves behavior implicit or underspecified.

7.2 Future Work

RFC 9114 [8] standardizes HTTP/3, which relies on a new transport protocol, QUIC, built on UDP instead of TCP to avoid head-of-line blocking. Compared to earlier versions, the security aspects of HTTP/3 remain relatively understudied. As future work, we plan to investigate the compliance of HTTP/3 implementations with respect

to RFC 9114 and compare the results with those of previous HTTP versions.

This study excludes TLS-related compliance rules, as the layered network stack design promotes layer independence. Consequently, HTTP/2 compliance should not be tied to the underlying transport or security layers. Future research could extend our methodology to include TLS-specific rules (e.g., protocol versions, cipher suites). Notably, variations in TLS implementations have been used to fingerprint phishing toolkits, which often rely on off-the-shelf TLS libraries [30]. Unlike production-grade proxies, Mitmproxy behaves differently when enforcing RFC 9113 compliance. Although its objectives differ, we observed 25 instances where payloads triggered behavior other than simple forwarding. Our findings suggest that compliance checks offer a promising direction for detecting and fingerprinting transparent proxies.

Preliminary results suggest that the timing of tests on cloud proxy instances affects HTTP/2 compliance, independent of network issues like packet loss. We omit this aspect from the paper due to the many potential contributing factors, leaving a deeper investigation for future work.

8 Conclusion

This paper investigates HTTP/2 compliance across 12 local and 3 cloud proxy implementations. Our results reveal discrepancies: although these proxies are generally stricter than their HTTP/1.1 counterparts, none is fully compliant with RFC 9113. In some cases, non-compliant messages are even forwarded to the final destination. Findings suggest that updates to specifications and improvements in how they are written can positively impact implementation compliance. Given prior attacks that exploit inconsistencies in the interpretation of HTTP messages, we argue that specifications should limit implementer flexibility. We strongly encourage the authors of future RFCs to enforce stricter requirements for intermediaries.

References

[1] Paul Adamczyk, Munawar Hafiz, and Ralph E Johnson. 2008. Non-compliant and proud: A case study of HTTP compliance. https://www.ideals.illinois.edu/items/11454. [Accessed: 26-Sep-2024].

[2] Mike Belshe, Roberto Peon, and Martin Thomson. 2015. Hypertext Transfer Protocol Version 2 (HTTP/2). RFC 7540. doi:10.17487/RFC7540

[3] Mike Belshe, Roberto Peon, Martin Thomson, and Alexey Melnikov. 2012. *SPDY Protocol.* Internet-Draft draft-ietf-httpbis-http2-00. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-httpbis-http2/00/ Work in Progress.

[4] Cory Benfield. 2025. GitHub - python-hyper/h2: Pure-Python HTTP/2 protocol implementation — github.com. https://github.com/python-hyper/h2. [Accessed 04-05-2025].

[5] Ilies Benhabbour, Mahmoud Attia, and Marc Dacier. 2025. HTTP Conformance vs. Middleboxes: Identifying Where the Rules Actually Break Down. In *International Conference on Passive and Active Network Measurement.* Springer-Verlag, Berlin, Heidelberg, 155–181.

[6] Ilies Benhabbour and Marc Dacier. 2022. NoPASARAN: a Novel Platform for Analysing Semi-Active elements in Routes Across a Network. *Applied Cybersecurity & Internet Governance* 1, 1 (2022), 1–25.

[7] Ilies Benhabbour and Marc Dacier. 2025. ENDEMIC: End-to-End Network Disruptions – Examining Middleboxes, Issues, and Countermeasures – A Survey. *ACM Comput. Surv.* 57, 7, Article 181 (Feb. 2025), 42 pages. doi:10.1145/3716372

[8] Mike Bishop. 2022. HTTP/3. RFC 9114. doi:10.17487/RFC9114

[9] Kevin Bock, Abdulrahman Alaraj, Yair Fax, Kyle Hurley, Eric Wustrow, and Dave Levin. 2021. Weaponizing Middleboxes for TCP Reflected Amplification. In *30th USENIX Security Symposium (USENIX Security 21).* USENIX Association, Vancouver, B.C., Canada, 3345–3361. https://www.usenix.org/conference/usenixsecurity21/presentation/bock

[10] Scott O. Bradner. 1997. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119. doi:10.17487/RFC2119

[11] Scott W. Brim and Brian E. Carpenter. 2002. Middleboxes: Taxonomy and Issues. RFC 3234. doi:10.17487/RFC3234

[12] Jianjun Chen, Jian Jiang, Haixin Duan, Nicholas Weaver, Tao Wan, and Vern Paxson. 2016. Host of Troubles: Multiple Host Ambiguities in HTTP Implementations. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (Vienna, Austria) *(CCS '16).* Association for Computing Machinery, New York, NY, USA, 1516–1527. doi:10.1145/2976749.2978394

[13] Akamai Cloud. 2021. Acceptable Use Policy — linode.com. https://www.linode.com/legal-aup/. [Accessed 30-04-2025].

[14] Piotr Duszynski. 2019. GitHub - drk1wi/Modlishka: Modlishka. Reverse Proxy. — github.com. https://github.com/drk1wi/Modlishka.

[15] Roy T. Fielding, Mark Nottingham, and Julian Reschke. 2014. Hypertext Transfer Protocol (HTTP/1.1): Caching. RFC 7234. doi:10.17487/RFC7234

[16] Roy T. Fielding, Mark Nottingham, and Julian Reschke. 2022. HTTP Semantics. RFC 9110. doi:10.17487/RFC9110

[17] Thiago Garrett, Ligia E Setenareski, Leticia M Peres, Luis CE Bona, and Elias P Duarte. 2018. Monitoring network neutrality: A survey on traffic differentiation detection. *IEEE Communications Surveys & Tutorials* 20, 3 (2018), 2486–2517.

[18] Omer Gil. 2017. Web cache deception attack. https://www.blackhat.com/docs/us-17/wednesday/us-17-Gil-Web-Cache-Deception-Attack-wp.pdf [Accessed: 26-Sep-2024].

[19] Kuba Gretzky. 2024. GitHub - kgretzky/evilginx2: Standalone man-in-the-middle attack framework used for phishing login credentials along with session cookies, allowing for the bypass of 2-factor authentication — github.com. https://github.com/kgretzky/evilginx2.

[20] Ilya Grigorik. 2013. Making the web faster with HTTP 2.0. *Commun. ACM* 56, 12 (2013), 42–49.

[21] Ronen Heled. 2005. HTTP REQUEST SMUGGLING. https://www.cgisecurity.com/lib/HTTP-Request-Smuggling.pdf [Accessed: 26-Sep-2024].

[22] Michio Honda, Yoshifumi Nishida, Costin Raiciu, Adam Greenhalgh, Mark Handley, and Hideyuki Tokuda. 2011. Is it still possible to extend TCP?. In *Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference* (Berlin, Germany) *(IMC '11).* Association for Computing Machinery, New York, NY, USA, 181–194. doi:10.1145/2068816.2068834

[23] Moto Ishizawa. 2020. GitHub - summerwind/h2spec: A conformance testing tool for HTTP/2 implementation. — github.com. https://github.com/summerwind/h2spec. [Accessed 15-09-2024].

[24] Bahruz Jabiyev, Anthony Gavazzi, Kaan Onarlioglu, and Engin Kirda. 2024. Gudifu: Guided Differential Fuzzing for HTTP Request Parsing Discrepancies. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses* (Padua, Italy) *(RAID '24).* Association for Computing Machinery, New York, NY, USA, 235–247. doi:10.1145/3678890.3678904

[25] Bahruz Jabiyev, Steven Sprecher, Anthony Gavazzi, Tommaso Innocenti, Kaan Onarlioglu, and Engin Kirda. 2022. FRAMESHIFTER: Security Implications of HTTP/2-to-HTTP/1 Conversion Anomalies. In *31st USENIX Security Symposium (USENIX Security 22).* USENIX Association, Boston, MA, 1061–1075. https://www.usenix.org/conference/usenixsecurity22/presentation/jabiyev

[26] Bahruz Jabiyev, Steven Sprecher, Kaan Onarlioglu, and Engin Kirda. 2021. T-Reqs: HTTP Request Smuggling with Differential Fuzzing. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21).* Association for Computing Machinery, New York, NY, USA, 1805–1820. doi:10.1145/3460120.3485384

[27] Ben Kallus, Prashant Anantharaman, Michael Locasto, and Sean Smith. 2024. The HTTP Garden: Discovering Parsing Vulnerabilities in HTTP/1.1 Implementations by Differential Fuzzing of Request Streams. doi:10.48550/arXiv.2405.17737

[28] James Kettle. 2019. HTTP Desync Attacks: Request Smuggling Reborn — portswigger.net. https://portswigger.net/research/http-desync-attacks-request-smuggling-reborn. [Accessed 10-03-2024].

[29] James Kettle. 2021. HTTP/2: The Sequel is Always Worse — portswigger.net. https://portswigger.net/research/http2. [Accessed 25-07-2024].

[30] Brian Kondracki, Babak Amin Azad, Oleksii Starov, and Nick Nikiforakis. 2021. Catching Transparent Phish: Analyzing and Detecting MITM Phishing Toolkits. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21).* Association for Computing Machinery, New York, NY, USA, 36–50. doi:10.1145/3460120.3484765

[31] Vlad Krasnov. 2016. HPACK: the silent killer (feature) of HTTP/2 — blog.cloudflare.com. https://blog.cloudflare.com/hpack-the-silent-killer-feature-of-http-2/. [Accessed 02-05-2025].

[32] Balachander Krishnamurthy and Martin Arlitt. 2001. PRO-COW: Protocol compliance on the web-a longitudinal study. In *Proceedings of the 3rd Conference on USENIX Symposium on Internet Technologies and Systems - Volume 3* (San Francisco, California) *(USITS'01).* USENIX Association, USA, 10.

[33] Seyed Ali Mirheidari, Sajjad Arshad, Kaan Onarlioglu, Bruno Crispo, Engin Kirda, and William Robertson. 2020. Cached and Confused: Web Cache Deception in the Wild. In *29th USENIX Security Symposium (USENIX Security 20).* USENIX Association, Boston, MA, USA, 665–682. https://www.usenix.org/conference/

usenixsecurity20/presentation/mirheidari
[34] Seyed Ali Mirheidari, Matteo Golinelli, Kaan Onarlioglu, Engin Kirda, and Bruno Crispo. 2022. Web Cache Deception Escalates!. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 179–196. https://www.usenix.org/conference/usenixsecurity22/presentation/mirheidari
[35] mitmproxy. 2025. mitmproxy - an interactive HTTPS proxy — mitmproxy.org. https://mitmproxy.org/. [Accessed 02-05-2025].
[36] Hoai Viet Nguyen, Luigi Lo Iacono, and Hannes Federrath. 2019. Mind the cache: large-scale explorative study of web caching. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing* (Limassol, Cyprus) *(SAC '19)*. Association for Computing Machinery, New York, NY, USA, 2497–2506. doi:10.1145/3297280.3297526
[37] Hoai Viet Nguyen, Luigi Lo Iacono, and Hannes Federrath. 2018. Systematic Analysis of Web Browser Caches. In *Proceedings of the 2nd International Conference on Web Studies* (Paris, France) *(WS.2 2018)*. Association for Computing Machinery, New York, NY, USA, 64–71. doi:10.1145/3240431.3240443
[38] Roberto Peon and Herve Ruellan. 2015. HPACK: Header Compression for HTTP/2. RFC 7541. doi:10.17487/RFC7541
[39] Cloudflare Radar. 2024. Adoption & Usage. https://radar.cloudflare.com/adoption-and-usage. [Accessed 19-11-2024].
[40] Cloudflare Radar. 2024. Overview. https://radar.cloudflare.com/. [Accessed 19-11-2024].
[41] Jannis Rautenstrauch and Ben Stock. 2024. Who's Breaking the Rules? Studying Conformance to the HTTP Specifications and its Security Impact. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) *(ASIA CCS '24)*. Association for Computing Machinery, New York, NY, USA, 843–855. doi:10.1145/3634737.3637678
[42] Jerome H Saltzer, David P Reed, and David D Clark. 1984. End-to-end arguments in system design. *ACM Transactions on Computer Systems (TOCS)* 2, 4 (1984), 277–288.
[43] Kaiwen Shen, Jianyu Lu, Yaru Yang, Jianjun Chen, Mingming Zhang, Haixin Duan, Jia Zhang, and Xiaofeng Zheng. 2022. HDiff: A Semi-automatic Framework for Discovering Semantic Gap Attack in HTTP Implementations. In *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, Baltimore, Maryland, USA, 1–13. doi:10.1109/DSN53405.2022.00014
[44] Ram Sundara Raman, Prerana Shenoy, Katharina Kohls, and Roya Ensafi. 2020. Censored Planet: An Internet-wide, Longitudinal Censorship Observatory. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, USA) *(CCS '20)*. Association for Computing Machinery, New York, NY, USA, 49–66. doi:10.1145/3372297.3417883
[45] Martin Thomson and Cory Benfield. 2022. HTTP/2. RFC 9113. doi:10.17487/RFC9113
[46] Giuseppe Trotta. 2024. GitHub - muraenateam/muraena: Muraena is an almost-transparent reverse proxy aimed at automating phishing and post-phishing activities. — github.com. https://github.com/muraenateam/muraena.
[47] Cong Wang, Xingliang Yuan, Yong Cui, and Kui Ren. 2017. Toward secure outsourced middlebox services: Practices, challenges, and beyond. *IEEE Network* 32, 1 (2017), 166–171.
[48] World Wide Web Consortium (W3C). 2001. Factory: HTTP Compliance and W3C QA. https://www.w3.org/2001/01/qa-ws/pp/alex-rousskov-measfact. [Accessed: 24-Jul-2024].

A Overlap Between HTTP/1.1 and HTTP/2 Test Cases

Table 7: Mapping of overlapping test cases between HTTP/1.1 and HTTP/2. The left column follows the numbering from Benhabbour et al. [5], while the right column shows our paper's numbering. A single HTTP/1.1 test can be derived into several HTTP/2 tests, as reflected in multiple entries in the right column.

HTTP/1.1 Test Case	HTTP/2 Test Case(s)
4	49, 50, 53, 54, 55, 56
6	134, 135, 138, 139, 140, 141
7	66, 67
8	150, 151
17	60, 145
44	51
45	136, 137
46	52

B List of Tests

Table 8: HTTP/2 client-side proxy tests, listing the requirement-level rules and their corresponding sections in RFC9113.

1	3.2	Client must send connection preface after TLS establishment
2	3.4	The connection preface starts with the string: PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n
3	3.4	Client preface must include a SETTINGS frame
4	4.1	Values greater than 16,384 MUST NOT be sent unless receiver has set larger SETTINGS_MAX_FRAME_SIZE
5	4.1	The frame type determines the format and semantics of the frame. Implementations MUST ignore and discard frames of unknown type.
6	4.1	A reserved 1-bit field. The semantics of this bit are undefined, and the bit MUST remain unset (0x00) when sending and MUST be ignored when receiving.
7	5.1	Receiving any frame other than HEADERS or PRIORITY on a stream in this (idle) state MUST be treated as a connection error of type PROTOCOL_ERROR.
8	5.1	If an endpoint receives additional frames, other than WINDOW_UPDATE, PRIORITY, or RST_STREAM, for a stream that is in the half-closed (remote) state, it MUST respond with a stream error of type STREAM_CLOSED.
9	5.1	An endpoint MUST NOT send frames other than PRIORITY on a closed stream.
10	5.1.1	Streams initiated by a client MUST use odd-numbered stream identifiers.
11	5.1.1	The identifier of a newly established stream MUST be numerically greater than all streams that the initiating endpoint has opened or reserved.
12	6.1	DATA frames MUST be associated with a stream.
13	6.1	If a DATA frame is received whose Stream Identifier field is 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
14	6.1	If a DATA frame is received whose stream is not in the 'open' or 'half-closed (local)' state, the recipient MUST respond with a stream error of type STREAM_CLOSED. (Tested in the idle state.)
15	6.1	If a DATA frame is received whose stream is not in the 'open' or 'half-closed (local)' state, the recipient MUST respond with a stream error of type STREAM_CLOSED. (Tested in the half-closed (remote) state.)
16	6.1	If a DATA frame is received whose stream is not in the 'open' or 'half-closed (local)' state, the recipient MUST respond with a stream error of type STREAM_CLOSED. (Tested in the closed state.)
17	6.2	A HEADERS frame without the END_HEADERS flag set MUST be followed by a CONTINUATION frame for the same stream.
18	6.2	If a HEADERS frame is received whose Stream Identifier field is 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
19	6.3	If a PRIORITY frame is received with a stream identifier of 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR
20	6.4	If a RST_STREAM frame is received with a stream identifier of 0x00, the recipient MUST treat this as a connection error of type PROTOCOL_ERROR.
21	6.4	A RST_STREAM frame with a length other than 4 octets MUST be treated as a connection error of type FRAME_SIZE_ERROR.
22	6.4	RST_STREAM frames MUST NOT be sent for a stream in the 'idle' state.
23	6.4	RST_STREAM frames MUST be associated with a stream.
24	6.5	When set, the ACK flag indicates that this frame acknowledges receipt and application of the peer's SETTINGS frame. When this bit is set, the frame payload of the SETTINGS frame MUST be empty.
25	6.5	The stream identifier for a SETTINGS frame MUST be zero (0x00).
26	6.5	A SETTINGS frame with a length other than a multiple of 6 octets MUST be treated as a connection error of type FRAME_SIZE_ERROR.
27	6.5.2	The initial value of SETTINGS_ENABLE_PUSH is 1. Any value other than 0 or 1 MUST be treated as a connection error of type PROTOCOL_ERROR.
28	6.5.2	For SETTINGS_INITIAL_WINDOW_SIZE, values above the maximum flow-control window size of $2^{31} - 1$ (2147483647) MUST be treated as a connection error of type FLOW_CONTROL_ERROR.
29	6.5.2	The value advertised by an endpoint MUST be between initial value ($2^{14} = 16,384$) and maximum allowed frame size ($2^{24} - 1 = 16,777,215$ octets), inclusive.
30	6.5.2	An endpoint that receives a SETTINGS frame with any unknown or unsupported identifier MUST ignore that setting.
31	6.5.3	Unsupported settings MUST be ignored.
32	6.7	If a PING frame is received with a Stream Identifier field value other than 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
33	6.9	A receiver MUST treat the receipt of a WINDOW_UPDATE frame with a flow-control window increment of 0 as a stream error of type PROTOCOL_ERROR.
34	6.9.1	A sender MUST NOT allow a flow-control window to exceed $2^{31} - 1$ octets. If a sender receives a WINDOW_UPDATE that causes a flow-control window to exceed this maximum, it MUST terminate either the stream or the connection, as appropriate.
35	6.9.1	The sender MUST NOT send a flow-controlled frame with a length that exceeds the space available in either of the flow-control windows advertised by the receiver.
36	6.10	If a CONTINUATION frame is received with a Stream Identifier field of 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
37	6.10	A CONTINUATION frame MUST be preceded by a HEADERS, PUSH_PROMISE or CONTINUATION frame without the END_HEADERS flag set. (Using HEADERS frame with END_HEADERS flag set)
38	6.10	CONTINUATION frames MUST be associated with a stream.
39	6.10	If the END_HEADERS flag is not set, this frame MUST be followed by another CONTINUATION frame. A receiver MUST treat the receipt of any other type of frame or a frame on a different stream as a connection error of type PROTOCOL_ERROR.
40	8.1	Trailers MUST NOT include pseudo-header fields.
41	8.1	Other frames (from any stream) MUST NOT occur between the HEADERS frame and any CONTINUATION frames that might follow.
42	8.1	An endpoint that receives a HEADERS frame without the END_STREAM flag set after receiving the HEADERS frame that opens a request or after receiving a final (non-informational) status code MUST treat the corresponding request or response as malformed.
43	8.2	Field names MUST be converted to lowercase when constructing an HTTP/2 message.
44	8.2.1	Field names MUST NOT contain control characters (0x00-0x1F)
45	8.2.1	Field names MUST NOT contain ASCII SP (0x20)
46	8.2.1	Field names MUST NOT contain DEL character (0x7F)
47	8.2.1	Field names MUST NOT contain high byte characters (0x80-0xFF)
48	8.2.1	With the exception of pseudo-header fields, which have a name that starts with a single colon, field names MUST NOT include a colon (ASCII COLON, 0x3a).
49	8.2.1	A field value MUST NOT contain line feed (ASCII LF, 0x0a). (Tested at the start of the value)
50	8.2.1	A field value MUST NOT contain carriage return (ASCII CR, 0x0d). (Tested at the start of the value)
51	8.2.1	A field value MUST NOT start with an ASCII whitespace character (ASCII SP or HTAB, 0x20 or 0x09).
52	8.2.1	A field value MUST NOT end with an ASCII whitespace character (ASCII SP or HTAB, 0x20 or 0x09).
53	8.2.1	A field value MUST NOT contain line feed (ASCII LF, 0x0a). (Tested at the middle of the value)
54	8.2.1	A field value MUST NOT contain line feed (ASCII LF, 0x0a). (Tested at the end of the value)
55	8.2.1	A field value MUST NOT contain carriage return (ASCII CR, 0x0d). (Tested at the middle of the value)
56	8.2.1	A field value MUST NOT contain carriage return (ASCII CR, 0x0d). (Tested at the end of the value)

57	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing connection header field.
58	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing proxy-connection header field.
59	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing keep-alive header field.
60	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing transfer-encoding header field.
61	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing upgrade header field.
62	8.2.2	The TE header field MAY be present in an HTTP/2 request; when it is, it MUST NOT contain any value other than 'trailers'.
63	8.3	Pseudo-header fields are not HTTP header fields. Endpoints MUST NOT generate pseudo-header fields other than those defined in this document.
64	8.3	Pseudo-header fields defined for responses MUST NOT appear in requests.
65	8.3	All pseudo-header fields sent from a client MUST appear in a field block before all regular field lines. Any request or response that contains a pseudo-header field that appears in a field block after a regular field line MUST be treated as malformed.
66	8.3	The same pseudo-header field name MUST NOT appear more than once in a field block. A field block for an HTTP request or response that contains a repeated pseudo-header field name MUST be treated as malformed. Tested with a request frame with the same value.
67	8.3	The same pseudo-header field name MUST NOT appear more than once in a field block. A field block for an HTTP request or response that contains a repeated pseudo-header field name MUST be treated as malformed. Tested with a request frame with different values.
68	8.3.1	Clients MUST NOT generate a request with a Host header field that differs from the ":authority" pseudo-header field.
69	8.3.1	':authority' MUST NOT include the deprecated userinfo subcomponent for "http" or "https" schemed URIs.
70	8.3.1	The ":path" pseudo-header field MUST NOT be empty for "http" or "https" URIs; "http" or "https" URIs that do not contain a path component MUST include a value of '/'.
71	8.3.1	All HTTP/2 requests MUST include exactly one valid value for the ":method", ":scheme", and ":path" pseudo-header fields, unless they are CONNECT requests. An HTTP request that omits mandatory pseudo-header fields is malformed. (Tested with :method missing)
72	8.3.1	All HTTP/2 requests MUST include exactly one valid value for the ":method", ":scheme", and ":path" pseudo-header fields, unless they are CONNECT requests. An HTTP request that omits mandatory pseudo-header fields is malformed. (Tested with :scheme missing)
73	8.3.1	All HTTP/2 requests MUST include exactly one valid value for the ":method", ":scheme", and ":path" pseudo-header fields, unless they are CONNECT requests. An HTTP request that omits mandatory pseudo-header fields is malformed. (Tested with :path missing)
74	8.4	A client cannot push. Thus, servers MUST treat the receipt of a PUSH_PROMISE frame as a connection error of type PROTOCOL_ERROR.
75	8.5	With the CONNECT method, the ":scheme" and ":path" pseudo-header fields MUST be omitted. (Tested with only :path present)
76	8.5	With the CONNECT method, the ":scheme" and ":path" pseudo-header fields MUST be omitted. (Tested with only :scheme present)
77	8.5	With the CONNECT method, the ":scheme" and ":path" pseudo-header fields MUST be omitted. (Tested with both present)
78	8.5	With the CONNECT method, The ":authority" pseudo-header field contains the host and port to connect to

Table 9: HTTP/2 server-side proxy tests, listing the requirement-level rules and their corresponding sections in RFC9113.

79	4.1	Values greater than 16,384 MUST NOT be sent unless receiver has set larger SETTINGS_MAX_FRAME_SIZE.
80	4.1	The frame type determines the format and semantics of the frame. Implementations MUST ignore and discard frames of unknown types.
81	4.1	A reserved 1-bit field. The semantics of this bit are undefined, and the bit MUST remain unset (0x00) when sending and MUST be ignored when receiving.
82	5.1	If this stream (initially in the idle state) is initiated by the server, then receiving a HEADERS frame MUST also be treated as a connection error of type PROTOCOL_ERROR.
83	5.1	An endpoint MUST NOT send any type of frame other than HEADERS, RST_STREAM, or PRIORITY in the reserved (local) state.
84	5.1	Receiving any frame other than HEADERS or PRIORITY on a stream in this (idle) state MUST be treated as a connection error of type PROTOCOL_ERROR.
85	5.1	If an endpoint receives additional frames, other than WINDOW_UPDATE, PRIORITY, or RST_STREAM, for a stream that is in the half-closed (remote) state, it MUST respond with a stream error of type STREAM_CLOSED.
86	5.1	An endpoint MUST NOT send frames other than PRIORITY on a closed stream.
87	5.1.1	Streams initiated by a server MUST use even-numbered stream identifiers.
88	5.4.2	An endpoint MUST NOT send a RST_STREAM in response to a RST_STREAM frame
89	6.1	DATA frames MUST be associated with a stream.
90	6.1	If a DATA frame is received whose Stream Identifier field is 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
91	6.1	If a DATA frame is received whose stream is not in the 'open' or 'half-closed (local)' state, the recipient MUST respond with a stream error of type STREAM_CLOSED. (Tested in the idle state.)
92	6.1	If a DATA frame is received whose stream is not in the 'open' or 'half-closed (local)' state, the recipient MUST respond with a stream error of type STREAM_CLOSED. (Tested in the half-closed (remote) state.)
93	6.1	If a DATA frame is received whose stream is not in the 'open' or 'half-closed (local)' state, the recipient MUST respond with a stream error of type STREAM_CLOSED. (Tested in the closed state.)
94	6.2	A HEADERS frame without the END_HEADERS flag set MUST be followed by a CONTINUATION frame for the same stream.
95	6.2	If a HEADERS frame is received whose Stream Identifier field is 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
96	6.3	If a PRIORITY frame is received with a stream identifier of 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
97	6.4	After receiving a RST_STREAM on a stream, the receiver MUST NOT send additional frames for that stream, except for PRIORITY
98	6.4	If a RST_STREAM frame is received with a stream identifier of 0x00, the recipient MUST treat this as a connection error of type PROTOCOL_ERROR.
99	6.4	A RST_STREAM frame with a length other than 4 octets MUST be treated as a connection error of type FRAME_SIZE_ERROR.
100	6.4	RST_STREAM frames MUST NOT be sent for a stream in the 'idle' state.
101	6.4	RST_STREAM frames MUST be associated with a stream.
102	6.5	When set, the ACK flag indicates that this frame acknowledges receipt and application of the peer's SETTINGS frame. When this bit is set, the frame payload of the SETTINGS frame MUST be empty.
103	6.5	The stream identifier for a SETTINGS frame MUST be zero (0x00).
104	6.5	A SETTINGS frame with a length other than a multiple of 6 octets MUST be treated as a connection error of type FRAME_SIZE_ERROR.
105	6.5.2	A server MUST NOT explicitly set this value (SETTINGS_ENABLE_PUSH) to 1. A server MAY choose to omit this setting (SETTINGS_ENABLE_PUSH) when it sends a SETTINGS frame, but if a server does include a value, it MUST be 0.
106	6.5.2	A server MUST NOT send a PUSH_PROMISE frame if it receives the SETTINGS_ENABLE_PUSH (0x02) parameter set to a value of 0.
107	6.5.2	For SETTINGS_INITIAL_WINDOW_SIZE, values above the maximum flow-control window size of $2^{31} - 1$ (2147483647) MUST be treated as a connection error of type FLOW_CONTROL_ERROR.
108	6.5.2	The value advertised by an endpoint MUST be between initial value ($2^{14} = 16,384$) and maximum allowed frame size ($2^{24} - 1 = 16,777,215$ octets), inclusive.
109	6.5.2	An endpoint that receives a SETTINGS frame with any unknown or unsupported identifier MUST ignore that setting.
110	6.5.3	Unsupported settings MUST be ignored.
111	6.6	If the Stream Identifier field of a PUSH_PROMISE frame specifies the value 0x00, a recipient MUST respond with a connection error of type PROTOCOL_ERROR.
112	6.6	The promised stream identifier MUST be a valid choice for the next stream sent by the sender. (Using Odd-numbered Stream ID (Invalid for Server))
113	6.6	The promised stream identifier MUST be a valid choice for the next stream sent by the sender. (Using Already Used Stream ID)
114	6.6	The promised stream identifier MUST be a valid choice for the next stream sent by the sender. (Using Lower Stream ID)
115	6.6	A PUSH_PROMISE frame without the END_HEADERS flag set MUST be followed by a CONTINUATION frame for the same stream.
116	6.7	If a PING frame is received with a Stream Identifier field value other than 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
117	6.9	A receiver MUST treat the receipt of a WINDOW_UPDATE frame with a flow-control window increment of 0 as a stream error of type PROTOCOL_ERROR.
118	6.9.1	A sender MUST NOT allow a flow-control window to exceed $2^{31} - 1$ octets. If a sender receives a WINDOW_UPDATE that causes a flow-control window to exceed this maximum, it MUST terminate either the stream or the connection, as appropriate.
119	6.9.1	The sender MUST NOT send a flow-controlled frame with a length that exceeds the space available in either of the flow-control windows advertised by the receiver.
120	6.10	A CONTINUATION frame MUST be preceded by a HEADERS, PUSH_PROMISE or CONTINUATION frame without the END_HEADERS flag set. (Using PUSH_PROMISE frame with END_HEADERS flag set)
121	6.10	If a CONTINUATION frame is received with a Stream Identifier field of 0x00, the recipient MUST respond with a connection error of type PROTOCOL_ERROR.
122	6.10	A CONTINUATION frame MUST be preceded by a HEADERS, PUSH_PROMISE or CONTINUATION frame without the END_HEADERS flag set. (Using HEADERS frame with END_HEADERS flag set)
123	6.10	CONTINUATION frames MUST be associated with a stream.
124	6.10	If the END_HEADERS flag is not set, this frame MUST be followed by another CONTINUATION frame. A receiver MUST treat the receipt of any other type of frame or a frame on a different stream as a connection error of type PROTOCOL_ERROR.
125	8.1	Trailers MUST NOT include pseudo-header fields.
126	8.1	Other frames (from any stream) MUST NOT occur between the HEADERS frame and any CONTINUATION frames that might follow.
127	8.1	An endpoint that receives a HEADERS frame without the END_STREAM flag set after receiving the HEADERS frame that opens a request or after receiving a final (non-informational) status code MUST treat the corresponding request or response as malformed.
128	8.2	Field names MUST be converted to lowercase when constructing an HTTP/2 message.
129	8.2.1	Field names MUST NOT contain control characters (0x00-0x1F)
130	8.2.1	Field names MUST NOT contain ASCII SP (0x20)
131	8.2.1	Field names MUST NOT contain DEL character (0x7F)
132	8.2.1	Field names MUST NOT contain high byte characters (0x80-0xFF)

133	8.2.1	With the exception of pseudo-header fields, which have a name that starts with a single colon, field names MUST NOT include a colon (ASCII COLON, 0x3a).
134	8.2.1	A field value MUST NOT contain line feed (ASCII LF, 0x0a). (Tested at the start of the value)
135	8.2.1	A field value MUST NOT contain carriage return (ASCII CR, 0x0d). (Tested at the start of the value)
136	8.2.1	A field value MUST NOT start with an ASCII whitespace character (ASCII SP or HTAB, 0x20 or 0x09).
137	8.2.1	A field value MUST NOT end with an ASCII whitespace character (ASCII SP or HTAB, 0x20 or 0x09).
138	8.2.1	A field value MUST NOT contain line feed (ASCII LF, 0x0a). (Tested at the middle of the value)
139	8.2.1	A field value MUST NOT contain line feed (ASCII LF, 0x0a). (Tested at the end of the value)
140	8.2.1	A field value MUST NOT contain carriage return (ASCII CR, 0x0d). (Tested at the middle of the value)
141	8.2.1	A field value MUST NOT contain carriage return (ASCII CR, 0x0d). (Tested at the end of the value)
142	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing connection header field.
143	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing proxy-connection header field.
144	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing keep-alive header field.
145	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing transfer-encoding header field.
146	8.2.2	An endpoint MUST NOT generate an HTTP/2 message containing upgrade header field.
147	8.2.2	The TE header field MAY be present in an HTTP/2 request; when it is, it MUST NOT contain any value other than 'trailers'.
148	8.3	Pseudo-header fields defined for requests MUST NOT appear in responses.
149	8.3	All pseudo-header fields sent from a server MUST appear in a field block before all regular field lines. Any request or response that contains a pseudo-header field that appears in a field block after a regular field line MUST be treated as malformed.
150	8.3	The same pseudo-header field name MUST NOT appear more than once in a field block. A field block for an HTTP request or response that contains a repeated pseudo-header field name MUST be treated as malformed. Tested with a response frame with the same value.
151	8.3	The same pseudo-header field name MUST NOT appear more than once in a field block. A field block for an HTTP request or response that contains a repeated pseudo-header field name MUST be treated as malformed. Tested with a response frame with different values.
152	8.3	Pseudo-header fields are not HTTP header fields. Endpoints MUST NOT generate pseudo-header fields other than those defined in this document.
153	8.3.2	For HTTP/2 responses, a single ":status" pseudo-header field is defined that carries the HTTP status code field. This pseudo-header field MUST be included in all responses, including interim responses; otherwise, the response is malformed.
154	8.4.1	The header fields in PUSH_PROMISE and any subsequent CONTINUATION frames MUST be a valid and complete set of request header fields.
155	8.4.1	In the PUSH_PROMISE frame, the server MUST include a method in the ":method" pseudo-header field that is safe and cacheable.
156	8.6	HTTP/2 does not support the 101 (Switching Protocols) informational status code.